

COMS5020A: Data Science in Practice

Introduction to Apache Spark

2023

1 What is it for?

Apache Spark is an open-source distributed computing system for processing and analyzing large-scale datasets. It offers a user-friendly API and supports various data processing tasks. Its key features include distributed computing, resilient distributed datasets (RDDs) for in-memory processing, data processing and transformation APIs, machine learning and advanced analytics with MLlib, graph processing with GraphX, stream processing with Spark Streaming, integration with big data tools, and built-in cluster management.

Overall, Apache Spark provides a unified and efficient framework for working with big data, making it suitable for diverse use cases ranging from batch processing to real-time analytics and machine learning.

2 But I already know sklearn!

Apache Spark and **scikit-learn** (sklearn) are both powerful tools for data processing and machine learning, but they have different strengths and use cases. Here are a few points to consider regarding the advantages of Apache Spark over scikit-learn:

1. **Scalability:** Apache Spark is designed for large-scale distributed computing, allowing processing of massive datasets that may not fit into the memory of a single machine. It offers efficient distributed computations across a cluster of machines, enabling parallel processing and faster execution times. Scikit-learn, on the other hand, is primarily designed for single-machine use and may not handle large datasets as efficiently.
2. **Distributed Computing:** Spark's distributed computing model performs computations in-memory, reducing disk I/O and accelerating data processing. This is advantageous for iterative algorithms or complex data transformations. Scikit-learn, being a single-machine library, may not provide the same level of performance for large-scale computations.
3. **Ecosystem and Integration:** Spark provides a comprehensive ecosystem with built-in libraries for SQL, streaming, graph processing, and machine learning (MLlib). This allows handling various aspects of data processing and analysis within a single framework. While scikit-learn is a robust machine learning library, its primary focus is on providing algorithms for machine learning tasks and does not offer the same breadth of capabilities as Spark.
4. **Real-time and Streaming Data:** Spark Streaming, a component of Apache Spark, enables real-time data processing and analysis of streaming data. It can process data streams continuously, making it suitable for use cases requiring real-time insights or immediate responses. Scikit-learn, however, is primarily focused on batch processing and may not have the same level of support for streaming data.

5. **Integration with Big Data Tools:** Apache Spark seamlessly integrates with other big data tools and frameworks such as Hadoop, Hive, HBase, and Kafka. This allows Spark to leverage data stored in these systems and utilize their processing capabilities. Scikit-learn, being a standalone library, does not have the same level of integration with big data tools.

In summary, Apache Spark is preferable when working with large-scale distributed datasets, requiring real-time or streaming processing, or leveraging the broader big data ecosystem. Scikit-learn is more suitable for single-machine tasks, traditional machine learning workflows, and scenarios where simplicity and ease of use are prioritized.

3 The keys to spark's success

Apache Spark can do a number of useful things for us that would otherwise be annoyingly hard to do:

1. **Cluster Setup:** Apache Spark operates on a cluster of machines, which can be set up using Spark's standalone cluster manager or integrated with other cluster managers like Apache Mesos or Hadoop YARN. The cluster consists of a master node and multiple worker nodes.
2. **Resilient Distributed Datasets (RDDs):** RDDs are the fundamental data abstraction in Spark. They are distributed collections of objects that can be processed in parallel across the nodes of the cluster. RDDs are immutable and fault-tolerant, meaning they can recover from failures. RDDs can be created from data stored in various sources, such as Hadoop Distributed File System (HDFS), local file systems, or other data storage systems.
3. **Transformations and Actions:** Spark offers a set of transformations and actions to manipulate and process RDDs. Transformations are operations that create a new RDD from an existing one, such as filtering, mapping, or aggregating the data. Transformations are lazy, meaning they are not executed immediately but rather build up a lineage graph to track the operations to be performed. Actions, on the other hand, trigger the execution of the transformations and return results or write data to an output.
4. **Distributed Execution:** When an action is called on an RDD, Spark creates a directed acyclic graph (DAG) of transformations based on the lineage graph. The DAG represents the computation plan for the desired output. Spark's Catalyst optimizer optimizes the execution plan to improve efficiency. The tasks in the DAG are then scheduled and executed in parallel across the worker nodes of the cluster. Data is exchanged between nodes as needed.
5. **Data Partitioning and Shuffling:** Spark automatically partitions the data in RDDs across the worker nodes based on the available resources. Partitioning allows Spark to parallelize the processing and ensures data locality by minimizing data movement across the network. In certain cases, when data needs to be shuffled across partitions, Spark performs efficient data shuffling to minimize network transfer and optimize performance.
6. **Fault Tolerance:** Spark provides fault tolerance through RDDs. If a node fails during the execution, Spark can reconstruct the lost RDD partitions by using the lineage graph. By keeping track of the transformations applied to the original data, Spark can recover lost data and resume the computation without starting from scratch.

7. **Built-in Libraries and APIs:** Spark offers high-level APIs in multiple programming languages like Scala, Java, Python, and R. It also provides various libraries for SQL queries (Spark SQL), machine learning (MLlib), graph processing (GraphX), and real-time streaming (Spark Streaming). These libraries provide ready-to-use functions, algorithms, and tools for common data processing and analysis tasks.

4 How do I get it?

The easiest way to install spark on your machine is to use pip. If you have already been using sklearn, you have it python already installed, and you probably have pip (the python package manager) installed already. To install sklearn, you can go to the command line and type:

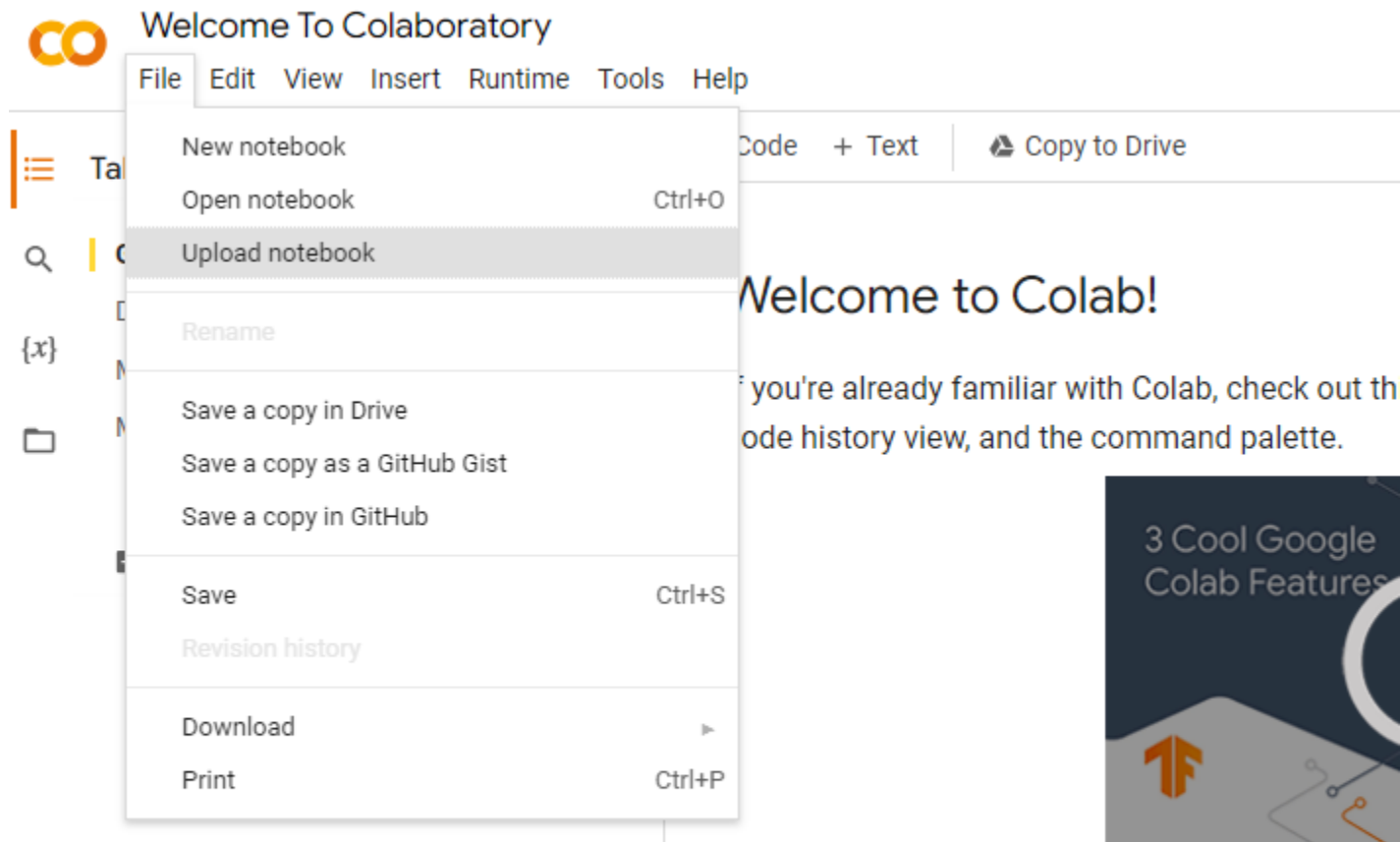
```
pip install pyspark
pip install pandas
```

If you do not want to install anything on your machine, but still want to use spark, you can use it through google colab (<https://colab.research.google.com/>)

**Note that you do not need to do this if you have done the pip install described above as you can then just run spark on your local machine, which will be easier.*

After you've signed in to your google account, you can go through the tutorial that is shown on screen if you are going to be using google colab for the spark exercises.

Next, to set up spark in colab, you can download the Spark.ipynb file from moodle and upload it to colab as shown below.



What is Colab?

Colab, or "Colaboratory", allows you to write and